

A prompt is a small work specification

WRITE IT — like a requirement

task and audience first · then only the requirements that resolve ambiguity

THE SKELETON — EVERY PROMPT

Purpose & audience: help [reader] make [decision] or complete [task].

Sources & scope: use [source / version / period]; include [scope]; exclude [items].

Selected requirements: only the types that resolve ambiguity for THIS artifact — a simple task needs a few, not all.

Acceptance evidence: check [specific result] against [source, calculation, or observable test].

Missing info & finish line: if [gap]: write “Not stated” and list it; if [conflict]: ask or stop. Answer, report limitations, then stop. Actions needing review: [actions].

REQUIREMENT TYPES — THE MENU (pick, don’t fill)

Functional behavior — what it must do: filter, calculate, extract, compare, explain.

Data & provenance — allowed inputs, definitions, period, traceability.

Scope & exclusions — in, out, and explicitly outside the task.

Audience & use — who uses it, for which decision.

Role & working behavior — a perspective plus observable conduct; a title alone is weak.

Content & completeness — required topics, fields, limitations.

Method & business rules — formulas, denominators, missing-data treatment.

Structure & interface — sections, tabs, slides, columns, controls.

Tone & presentation — voice, reading level, units, formats.

Accessibility & usability — labels, contrast, keyboard, understandable language.

Quality & acceptance — observable correctness and completeness criteria.

Constraints & authority — allowed tools, approvals, stop conditions.

Delivery & compatibility — file type, editability, offline use, destination.

Maintenance & reuse — refresh procedure, versions, reusable parameters.

REQUIREMENT QUALITY — IS EACH ONE...

necessary · clear · complete enough · consistent · feasible · singular · **verifiable**. A requirement you cannot check is a wish. Separate required outcomes from preferences; make trade-offs explicit.

BEFORE / AFTER — THE SUPPLIER CASE

Before: “Analyze the supplier data and tell me which supplier is worst.” — the model picks the rows, the metric, and “worst” for you, silently.

After — inspect: “Before calculating anything, inspect the file. Tell me what one row represents, how many detail rows it contains, whether any values are missing, and whether any total or summary row could be counted twice. Tell me what you would include or exclude, then wait.”

After — analyze: “Using detail rows only, calculate each supplier’s return rate as total Units_Returned divided by total Units_Shipped. Rank suppliers from highest to lowest. Show the totals used, reconcile the overall totals to the source file, and state one limitation of the comparison.”

Why it wins: each added line is a requirement type fixing one named weakness — and nothing is calculated before the scope is agreed.

RUN IT — the course tasks, one habit

produce the artifact, then check it against its own acceptance evidence

THE COURSE TASKS (deck order · workbook tab)

Analyze data — inspect first, then wait; return rate from totals, detail rows only; rank, reconcile to the source, state one limitation; follow-up = findings, not causes · 1-Analyze-Data.

Excel charts — the returned workbook: Summary sheet, native editable charts, traceable formulas, totals reconciled before it returns · Excel-Charts.

Build a dashboard — describe the behavior: date range, supplier and site filters, metric selector, trend chart, sortable table, reset; every filter updates every view · 2-Build-Dashboard.

Present findings — five slides, one main message each; findings separated from recommendations; no invented causes or commitments · 3-Present-Findings.

Summarize email — current status; decisions WITH conditions; owners and due dates; “Not stated” for gaps; cite sender and date; send nothing · 4-Summarize-Email.

Extract quotes — values verbatim, original currencies, “Not stated” when absent, source file and page per value; no ranking yet · Quote-Extract.

Compare quotes — normalize only with a supplied basis; formulas visible; no best quote until the basis is complete · Quote-Compare.

Self-study — research workbook (Research) · explain a topic clearly (5-Explain-Clearly) — same discipline; Reference Deck.

ACCEPTANCE CHECKS — BEFORE YOU TRUST IT

The loop — PLAN the task and the success check · DO run the prompt against the supplied source · CHECK the result against evidence or a known calculation · ACT: fix the specific failure and rerun the affected check.

Trace — one output number back to its source records.

Recompute — one calculation independently (code or by hand).

Reconcile — totals against a number you already trust.

Exercise — every requested behavior once (filter, sort, reset, empty case).

Coverage — compare against the requirement list; missing = not done.

Read as the reader — can the audience act on it? That is validation.

Never accept “looks done”: ask for evidence — the test it ran and what it returned.

ASK vs DELEGATE

ASK — request an answer or draft; you read, check, decide. The specification above is enough.

DELEGATE — assign bounded work: add the inputs register, the checks that stop the run, permissions, gates for irreversible steps, and the finished-report format.

The boundary: a prompt requests behavior; configured permissions and approvals restrict available actions. Reversible? proceed. Irreversible? ask.

SAVE WHAT WORKS

Each reusable prompt: name · owner · version · the example that proved it · its acceptance check.

Where: the Course Workbook carries every course prompt in a named tab, in deck order; the Prompt Template Creator (one HTML file) offers the same task families, plus explain-a-topic, with their requirement menus.

Refresh: facts about products and models expire — date them and re-verify before big reuse.